

به نام پروردگار هدایت کننده به راه راست



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

پروژه درس آزمون نرم افزار

تحلیل و توضیح کد اجرایی ارزیابی مدل های زبانی بزرگ

در تولید تست کیس ها با استفاده از بنچمارک TestEval

استاد : جناب آقای دکتر محمدرضا شهرباف

محدثه آخوندی

4003613002

خرداد ماه 1404

به صورت کامل کدی که اجرا کردم را شرح میدهم:

```
!git clone https://github.com/LLM4SoftwareTesting/TestEval.git
%cd TestEval
```

در این سلول، ابتدا مخزن GitHub مربوط به پروژه TESTEVAL کلون می‌شود. این مخزن شامل کدها، مجموعه داده‌ها و اسکریپت‌های لازم برای اجرای آزمایش‌های مقاله است. دستور `git clone` برای کپی کل مخزن از GitHub به فضای فعلی Colab استفاده می‌شود.

پس از کلون شدن، با استفاده از `%cd TestEval` مسیر کاری جاری به دایرکتوری `TestEval` تغییر می‌کند تا اجرای کدهای بعدی در همان دایرکتوری انجام شود.

```
import textwrap
print(textwrap.fill("پیچیده شده textwrap این یک جمله نمونه است که با", width=20))
```

در این سلول، ماژول `textwrap` از کتابخانه استاندارد پایتون ایمپورت شده است. هدف از این ماژول، تنظیم و شکستن متن‌های طولانی به خطوط کوچکتر با عرض مشخص است. تابع `fill()` متن فارسی داده‌شده را به خطوطی با عرض حداکثر ۲۰ کاراکتر تقسیم می‌کند.

برای تست وجود این کتابخانه در کولب این کد را نوشتیم زیرا این یک کتابخانه `built-in` هست و نباید در فایل `requirements` باشد پس از آن فایل در بخش بعدی حذف می‌کنیم.

```
!sed -i '/textwrap/d' requirements.txt
```

در این سلول، با استفاده از دستور `sed` که یک ابزار خط فرمان در لینوکس است، خطی که شامل عبارت `textwrap` است از فایل `requirements.txt` حذف می‌شود. این کار زمانی انجام می‌شود که بخواهیم مطمئن شویم پکیجی که به طور پیش‌فرض در پایتون وجود دارد مانند `textwrap` به اشتباه در لیست نصب قرار نگرفته باشد.

```
!pip install -r requirements.txt
```

در این سلول، تمامی کتابخانه‌هایی که در فایل `requirements.txt` تعریف شده‌اند، با استفاده از `pip` نصب می‌شوند. این مرحله برای اطمینان از فراهم بودن تمامی پیش‌نیازهای پروژه بسیار حیاتی است.

```
import os
os.environ["GOOGLE_API_KEY"] = "AIzaSyC4jzgSxl6fCDPZLGmGeTz8cWutO3bCTq8"
```

در این سلول، کلید API مربوط به Google Gemini در متغیرهای محیطی ذخیره می‌شود تا در مراحل بعدی اجرای مدل از آن استفاده شود. این روش از ذخیره‌سازی کلید به صورت امن در محیط اجرای کد مرسوم است و باعث می‌شود کلید به صورت مستقیم در خروجی چاپ نشود یا در فایل‌ها ذخیره نگردد.

```
!sed -i 's|LC_data/leetcode-bench-py.jsonl|data/leetcode-py.jsonl|g'  
generate_cov_gemini.py
```

این سلول با استفاده از دستور `sed`، مسیر فایل داده در اسکریپت `generate_cov_gemini.py` را به مسیر جدید تغییر می‌دهد. این کار برای تطبیق اسکریپت با داده‌های موجود در پروژه فعلی انجام می‌شود تا هنگام اجرا ارجاع به مسیرهای اشتباه صورت نگیرد.

```
!mkdir -p predictions
```

در این سلول یک پوشه با نام `predictions` ایجاد می‌شود تا فایل‌های خروجی مانند تست کیس‌های تولیدشده در آن ذخیره شوند. استفاده از `-p` تضمین می‌کند که اگر پوشه قبلاً وجود داشته باشد، خطایی رخ ندهد.

```
import json  
  
with open("data/leetcode-py.jsonl", "r") as f:  
    lines = f.readlines()  
  
subset = lines[:10]  
  
with open("data/leetcode-py-small.jsonl", "w") as f:  
    f.writelines(subset)
```

در این سلول، هدف ایجاد نسخه‌ای سبک‌تر و کوچک‌تر از مجموعه داده اصلی `leetcode-py.jsonl` است تا بتوان آزمایش‌های سریع‌تری انجام داد.

```
!sed -i 's|data/leetcode-py.jsonl|data/leetcode-py-small.jsonl|g'  
generate_cov_gemini.py
```

این خط مسیر فایل داده‌ی اصلی `data/leetcode-py.jsonl` را با مسیر فایل کوچک‌شده `data/leetcode-py-small.jsonl` در داخل فایل `generate_cov_gemini.py` جایگزین می‌کند. هدف از این کار، اجرای اسکریپت بر روی مجموعه داده‌ی کم‌حجم‌تر برای تست سریع‌تر و سبک‌تر است.

```
!python generate_cov_gemini.py --model models/gemini-1.5-flash-latest --  
num_tests 1
```

در این خط، اسکریپت `generate_cov_gemini.py` اجرا می‌شود تا مدل زبانی `gemini-1.5-flash-latest` برای هر مسئله برنامه‌نویسی تنها یک تست کیس تولید کند. این اجرای سبک با استفاده از مجموعه داده‌ی کوچک‌تر (که قبلاً تنظیم شد) انجام می‌شود و برای بررسی سریع عملکرد تولید تست مدل استفاده می‌گردد.

```
!python format.py --mode overall --path totalcov_gemini-1.5-flash-  
latest.jsonl
```

این خط، اسکریپت `format.py` را با حالت `overall` اجرا می‌کند تا فایل خروجی مدل `totalcov_gemini-1.5-flash-latest.jsonl` را فرمت‌بندی و خوانا تر کند.

در اصل، این اسکریپت برای زیباسازی و یکسان‌سازی قالب کدهای تولیدشده توسط مدل به کار می‌رود. خروجی آن، یک فایل جدید با نام مشابه اما با پسوند `format.jsonl` است که تست‌کیس‌ها در آن به‌صورت خوانا، تمیز، و مطابق با استانداردهای کدنویسی پایتون قالب‌بندی شده‌اند.

```
!python eval_overall.py --path totalcov_gemini-1.5-flash-latestformat.jsonl
```

این دستور اسکریپت `eval_overall.py` را اجرا می‌کند تا عملکرد تست‌کیس‌های تولیدشده توسط مدل را به‌صورت کلی (overall) ارزیابی کند.

ورودی آن، فایلی است که قبلاً با فرمت مناسب ذخیره شده (`totalcov_gemini-1.5-flash-latestformat.jsonl`) و شامل کدهای تست‌شده و نتایج اجرای آن‌ها است.

اسکریپت با پردازش این داده‌ها، معیارهایی مانند نرخ موفقیت تست‌ها، پوشش کد (code coverage)، میزان خطا و کیفیت تست‌کیس‌ها را محاسبه کرده و گزارش نهایی از کیفیت عملکرد مدل ارائه می‌دهد.

خروجی آن:

```
totalcov_gemini-1.5-flash-latestformat.jsonl
[1, 2, 5]
10
10it [00:47, 4.75s/it]
Syntax Correctness: 1.0
Executable Correctness: 1.0
line coverage@1 0.8843248689844435
branch coverage@1 0.8106944444444444
line coverage@2 0.8843248689844435
branch coverage@2 0.8106944444444444
line coverage@5 0.8843248689844435
branch coverage@5 0.8106944444444444
Average Line Coverage: 0.8843248689844435, Average Branch Coverage: 0.8106944444444444
```

Syntax Correctness: 1.0

نشان می‌دهد که تمام تست‌کیس‌های تولیدشده از نظر نحوی (Syntax) صحیح بوده‌اند و هیچ خطای نوشتاری نداشته‌اند.

Executable Correctness: 1.0

یعنی تمام تست‌ها بدون خطا اجرا شده‌اند و کد تولیدشده به‌درستی قابل اجرا بوده است.

line coverage@1 / @2 / @5 = 0.8843...

بیانگر میزان پوشش خطوط کد (Line Coverage) در تست‌هایی است که به ترتیب 1، 2 و 5 مورد از آن‌ها تولید شده. همه مقادیر حدود 88.4٪ هستند، یعنی تست‌ها به‌طور متوسط نزدیک به ۹۰٪ خطوط کد را پوشش داده‌اند.

branch coverage@1 / @2 / @5 = 0.8106...

این اعداد میزان پوشش شاخه‌ها (شرط‌ها و if‌ها) را نشان می‌دهد که حدود 81٪ از شاخه‌های منطقی در کد پوشش داده شده‌اند.

Average Line Coverage

میانگین کلی پوشش خطوط کد توسط تست‌ها : $0.8843 \approx 88.4\%$

Average Branch Coverage

میانگین کلی پوشش شاخه‌ها توسط تست‌ها : $0.8106 \approx 81.1\%$

```
!python eval_base.py --path totalcov_gemini-1.5-flash-latestformat.jsonl
```

این دستور اسکریپت `eval_base.py` را اجرا می‌کند تا فایل تست‌ها را بدون در نظر گرفتن هدف خاصی (مثل خط یا مسیر خاص) فقط بر اساس موفقیت در اجرا و میزان پوشش پایه‌ای (baseline) بررسی کند؛ یعنی ارزیابی ساده و عمومی عملکرد تست‌ها انجام می‌دهد.

خروجی :

```
10
0it [00:00, ?it/s]covered line 16
covered line 29
line not covered
covered line 32
line not covered
covered line 36
covered branch 15-16
covered branch 28-32
branch not covered
covered branch 31-32
branch not covered
covered branch 35-36
1it [00:01, 1.16s/it]covered line 23
covered line 28
covered line 29
covered line 30
covered line 32
covered branch 22-23
covered branch 27-30
covered branch 31-32
2it [00:02, 1.11s/it]line not covered
covered line 22
covered line 29
covered line 30
covered line 31
covered line 32
line not covered
covered line 34
line not covered
covered line 37
```

covered line 39
branch not covered
covered branch 21-22
covered branch 28-35
covered branch 36-37
covered branch 38-39
3it [00:03, 1.10s/it]line not covered
line not covered
line not covered
line not covered
covered line 32
branch not covered
branch not covered
covered branch 31-32
4it [00:04, 1.09s/it]line not covered
branch not covered
5it [00:05, 1.11s/it]line not covered
line not covered
line not covered
line not covered
covered line 27
line not covered
covered line 29
covered line 30
line not covered
line not covered
line not covered
covered line 36
line not covered
covered line 38
branch not covered
branch not covered

14

branch not covered
covered branch 26-30
branch not covered
branch not covered
covered branch 35-38
branch not covered
branch not covered
6it [00:06, 1.10s/it]covered line 21
covered line 22
covered line 27
line not covered
line not covered
line not covered
branch not covered
branch not covered
covered branch 20-22
covered branch 26-27
7it [00:07, 1.09s/it]line not covered
branch not covered
8it [00:08, 1.10s/it]covered line 18
line not covered
covered line 37
covered line 39
covered line 41
covered line 42
covered line 46
line not covered
covered line 55
covered line 56
line not covered
covered line 61
covered line 62
covered line 63

branch not covered

```
branch not covered
branch not covered
covered branch 54-56
branch not covered
covered branch 17-18
covered branch 45-46
covered branch 60-63
covered branch 36-37
covered branch 38-39
covered branch 40-42
git [00:09, 1.09s/it]line not covered
covered line 24
covered line 25
covered line 26
covered line 34
covered line 36
covered line 43
covered line 45
branch not covered
covered branch 23-26
covered branch 33-34
covered branch 35-36
covered branch 42-43
covered branch 44-45
covered branch 24-26
10it [00:11, 1.14s/it]
10 10 10
Syntax Correctness: 1.0
Executable Correctness: 1.0
Accuracy in cover selected line: 0.6197183098591549
Accuracy in cover selected branch: 0.5918367346938775
Easy branch coverage rate: 0.9166666666666666
Medium branch coverage rate: 0.5
Hard branch coverage rate: 0.48
```

این خروجی مربوط به ارزیابی عملکرد مدل در حالتی است که فقط تلاش می‌شود به‌طور کلی خطوط یا شاخه‌های کد پوشش داده شود، بدون اینکه هدف خاصی (مثل رسیدن به یک خط مشخص یا مسیر خاص) تعیین شده باشد.

در این ارزیابی:

1. Syntax Correctness: 1.0

همه تست‌ها از نظر نحوی صحیح هستند (هیچ کد اشتباهی تولید نشده).

2. Executable Correctness: 1.0

تمام تست‌ها بدون خطای زمان اجرا اجرا شده‌اند.

3. Accuracy in cover selected line: 0.62

حدود ۶۲٪ از خطوط هدف که مدل باید پوشش می‌داد، با موفقیت پوشش داده شده‌اند.

4. Accuracy in cover selected branch: 0.59

حدود ۵۹٪ از شاخه‌ها یا مسیرهای شرطی موردنظر پوشش داده شده‌اند.

مدل در پوشش شاخه‌های ساده عملکرد بسیار خوبی داشته و بیش از ۹۱٪ آن‌ها را پوشش داده است.

در شاخه‌های با پیچیدگی متوسط، عملکرد مدل به حدود ۵۰٪ رسیده است.

در شاخه‌های دشوار (که نیازمند استدلال پیچیده هستند)، مدل توانسته تنها ۴۸٪ از آن‌ها را پوشش دهد.

این دستور اسکریپت `reval_pathcov_base.py` اجرا می‌کند تا توانایی مدل در پوشش مسیرهای اجرایی خاص (**path**)

متوالی در کد را با موفقیت طی کنند یا خیر. این سنجش پیچیده‌تر از پوشش خط یا شاخه تکی است.

[illegible]

```
[
'LOOP #1: Entered for loop at line 21-39\n', 'BRANCH #8: Covered else branch at line 35-38'\n', 'LOOP #1: Entered for loop at line 21-39\n', '↑ ↓ ↕ ☰ ⚙️ 🗑️ 🔍 : nc  

'LOOP #1: Entered for loop at line 21-39\n', 'BRANCH #8: Covered else branch at line 35-38'\n', 'LOOP #1: Entered for loop at line 21-39\n', '☰ ⚙️ 🗑️ 🔍 : nc  

'LOOP #1: Entered for loop at line 21-39\n', 'BRANCH #8: Covered else branch at line 35-38'\n', 'LOOP #1: Entered for loop at line 21-39\n', 'BRANCH #4: Covered elif branc  

'LOOP #1: Entered for loop at line 21-39\n', 'BRANCH #8: Covered else branch at line 35-38'\n', 'LOOP #1: Entered for loop at line 21-39\n', 'BRANCH #4: Covered elif branc  

git [00:00, 54.35it/s] 'LOOP #1: Entered for loop at line 18-23\n', 'LOOP #2: Entered for loop at line 19-23\n', 'BRANCH #1: Covered if branch at line 20-22\n', 'LOOP #2:  

'LOOP #1: Entered for loop at line 18-23\n', 'LOOP #2: Entered for loop at line 19-23\n', 'BRANCH #1: Covered if branch at line 20-22\n', 'LOOP #2: Entered for loop at li  

'LOOP #1: Entered for loop at line 18-23\n', 'LOOP #2: Entered for loop at line 19-23\n', 'BRANCH #1: Covered if branch at line 20-22\n', 'LOOP #2: Entered for loop at li  

'LOOP #1: Entered for loop at line 18-23\n', 'LOOP #2: Entered for loop at line 19-23\n', 'BRANCH #1: Covered if branch at line 20-22\n', 'LOOP #2: Entered for loop at li  

'LOOP #1: Entered for loop at line 18-23\n', 'LOOP #2: Entered for loop at line 19-23\n', 'BRANCH #1: Covered if branch at line 20-22\n', 'LOOP #2: Entered for loop at li  

[ 'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line  

'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line  

'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line 19-20\n', 'LOOP #1: Entered for loop at line  

'LOOP #2: Entered while loop at line 30-47\n', 'LOOP #3: Entered for loop at line 33-47\n', 'LOOP #4: Entered for loop at line 35-44\n', 'LOOP #1: Entered for loop at lin  

'LOOP #2: Entered while loop at line 30-47\n', 'LOOP #3: Entered for loop at line 33-47\n', 'LOOP #4: Entered for loop at line 35-44\n', 'LOOP #1: Entered for loop at lin  

'LOOP #2: Entered while loop at line 30-47\n', 'LOOP #3: Entered for loop at line 33-47\n', 'LOOP #4: Entered for loop at line 35-44\n', 'LOOP #1: Entered for loop at lin  

[ 'LOOP #1: Entered for loop at line 21-27\n', 'LOOP #2: Entered for loop at line 22-27\n', 'BRANCH #2: Covered if branch at line 23-26\n', 'LOOP #2: Entered for loop at li  

'LOOP #1: Entered for loop at line 21-27\n', 'LOOP #2: Entered for loop at line 22-27\n', 'BRANCH #2: Covered if branch at line 23-26\n', 'LOOP #2: Entered for loop at li  

'LOOP #1: Entered for loop at line 21-27\n', 'LOOP #2: Entered for loop at line 22-27\n', 'BRANCH #2: Covered if branch at line 23-26\n', 'LOOP #2: Entered for loop at li  

10it [00:00, 19.82it/s]  

Syntax Correctness: 1.0  

Executable Correctness: 1.0  

10 10 22 34  

path exact match accuracy: 0.6470588235294118  

path similarity score: 0.8284313725490197
```

این اسکریپت عملکرد مدل را از منظر پوشش مسیرهای منطقی (Path Coverage) بررسی می‌کند؛ یعنی بررسی می‌شود آیا تست‌ها توانسته‌اند توالی‌های مشخص از تصمیم‌ها و شرط‌ها مثل if-else های پشت‌سرهم را در برنامه بپوشانند یا نه.

مهم‌ترین نتایج خروجی:

1. Syntax Correctness: 1.0

تمامی تست‌کیس‌ها از نظر نحوی درست هستند.

2. Executable Correctness: 1.0

همه تست‌ها با موفقیت اجرا شده‌اند.

3. Accuracy in cover selected path: 0.51

مدل تنها ۵۱٪ از مسیرهای مورد نظر را با تست‌های تولیدشده پوشش داده است. این نشان می‌دهد که پوشش مسیرهای ترکیبی (که شامل چندین شرط است) برای مدل چالش‌برانگیزتر است.

4. path coverage rate = 0.7802197802197802

یعنی به‌طور کلی حدود ۷۸٪ از مسیرهای مختلف در کد توسط تست‌ها پوشش داده شده‌اند.

توجه

به علت محدودیت‌های استفاده از مدل‌ها در نسخه رایگان فقط ده تا از 210 برنامه موجود پایتون برای تست استفاده شده پس دقت‌ها به اندازه کافی نیستند ولی برای تست برنامه و اجرای آن کفایت می‌کردند.